

STAT462 Assignment 2

Thomas Waldin - 17775654

2025-08-30

The objective of this report is to investigate datasets and demonstrate understanding of classification algorithms.

Part A: Pumpkin Seeds

A logistic regression was trained for automatic classification of two types of pumpkin seeds, Çerçvelik and Ürgüp Sivrisi, based on geometric (measured) features of these seeds.

1)

The datasets `seeds_training.csv` and `seeds_test.csv` were loaded. Since the class labels (the seed types) contain characters the class features were encoded as a factor variable in both training and test set.

```
# Import libraries
library(tidyverse)
library(caret)
library(MASS)
library(ggplot2)

# Read data
df_train <- read_csv("seeds_training.csv")
df_test  <- read_csv("seeds_test.csv")

# Encode Class variable as factor
seeds_train <- df_train %>%
  mutate(Class = as.factor(Class))

seeds_test <- df_test %>%
  mutate(Class = as.factor(Class))
```

2)

A multiple logistic regression classifier was fit to the data using all of the features. The accuracy on the test set was found to be 0.874.

```

# Fit a multiple logistic regression using all features
lm <- glm(Class ~ .,
           data = seeds_train,
           family = binomial)

# Make predictions on test set
pred_prob <- predict(lm,
                     newdata = seeds_test,
                     type = "response")

# Convert probabilities to class predictions (using 0.5 as threshold)
class_labels <- levels(as.factor(seeds_test$Class))
pred_class <- ifelse(pred_prob > 0.5, class_labels[2], class_labels[1])

# Calculate and print accuracy
accuracy <- mean(pred_class == seeds_test$Class)
cat("Accuracy on test set:", round(accuracy, 4), "\n")

```

```
## Accuracy on test set: 0.874
```

3)

A confusion matrix for the model was generated based on its predictions on the test set. The concept of sensitivity and specificity do not make a lot of sense in this example. Neither seed is inherently positive or negative, each are just different varieties, and the choice of which seed being positive is arbitrary. Without further context it is unknown if one misclassification is more problematic than the other, and it is likely that it is equally important to classify each correctly. The accuracy is likely a better metric in this case.

```

# Compute confusion matrix
confusionMatrix(as.factor(pred_class), as.factor(seeds_test$Class), positive = "Cercevelik")

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction      Cercevelik Urgup Sivrisi
## Cercevelik           242         44
## Urgup Sivrisi         19        195
##
##              Accuracy : 0.874
##              95% CI : (0.8417, 0.9018)
##      No Information Rate : 0.522
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.7464
##
## Mcnemar's Test P-Value : 0.002497
##
##              Sensitivity : 0.9272
##              Specificity : 0.8159
##      Pos Pred Value : 0.8462
##      Neg Pred Value : 0.9112
##      Prevalence : 0.5220

```

```
##          Detection Rate : 0.4840
##    Detection Prevalence : 0.5720
##      Balanced Accuracy : 0.8716
##
##      'Positive' Class : Cercevelik
##
```

4)

Observing the summary of the GLM model, it appeared that the area, the minor axis length, and the equivalent diameter of the seeds were the most significant features.

```
summary(lm)
```

```
##
## Call:
## glm(formula = Class ~ ., family = binomial, data = seeds_train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   214.593346  204.305124   1.050 0.293554
## Area           0.005708   0.002214   2.579 0.009923 **
## Perimeter      0.017973   0.062029   0.290 0.772008
## Major_Axis_Length 0.264676  0.115440   2.293 0.021862 *
## Minor_Axis_Length 0.515927  0.141661   3.642 0.000271 ***
## Convex_Area    -0.004525   0.002210  -2.048 0.040585 *
## Equiv_Diameter -1.370730   0.231176  -5.929 3.04e-09 ***
## Eccentricity    7.366701  34.013086   0.217 0.828532
## Solidity      -78.846626 178.684652  -0.441 0.659024
## Extent         0.132721   1.494525   0.089 0.929237
## Roundness      3.469268  46.721137   0.074 0.940808
## Aspect_Ration  -4.298850   7.201718  -0.597 0.550561
## Compactness   -68.374384  91.412640  -0.748 0.454475
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 2769.5  on 1999  degrees of freedom
## Residual deviance: 1130.6  on 1987  degrees of freedom
## AIC: 1156.6
##
## Number of Fisher Scoring iterations: 7
```

5)

Another GLM model was trained, only using the identified three most significant features. The accuracy and confusion matrix were generated so that they could be compared with the previously trained model. The accuracy on the (arbitrary) positive class was marginally better in the second model (0.878 vs. 0.874). As previously mentioned, the accuracy would be a good metric to consider in this case, so the model with just three features should be selected. There is likely a model between these two, with more of the significant features, but not all features which may outperform both models that were considered.

```

# Fit a multiple logistic regression using select features
lm <- glm(Class ~ Area + Minor_Axis_Length + Equiv_Diameter,
          data = seeds_train,
          family = binomial)

# Make predictions on test set
pred_prob <- predict(lm,
                    newdata = seeds_test,
                    type = "response")

# Convert probabilities to class predictions (using 0.5 as threshold)
class_labels <- levels(as.factor(seeds_test$Class))
pred_class <- ifelse(pred_prob > 0.5, class_labels[2], class_labels[1])

# Calculate and print accuracy
accuracy <- mean(pred_class == seeds_test$Class)
cat("Accuracy on test set:", round(accuracy, 4), "\n", "\n")

```

```

## Accuracy on test set: 0.878
##

```

```

# Compute confusion matrix
confusionMatrix(as.factor(pred_class), as.factor(seeds_test$Class), positive = "Cercevelik")

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction      Cercevelik Urgup Sivrisi
##   Cercevelik           237           37
##   Urgup Sivrisi           24           202
##
##              Accuracy : 0.878
##              95% CI : (0.8461, 0.9054)
##   No Information Rate : 0.522
##   P-Value [Acc > NIR] : <2e-16
##
##              Kappa : 0.755
##
##   McNemar's Test P-Value : 0.1244
##
##              Sensitivity : 0.9080
##              Specificity : 0.8452
##              Pos Pred Value : 0.8650
##              Neg Pred Value : 0.8938
##              Prevalence : 0.5220
##              Detection Rate : 0.4740
##   Detection Prevalence : 0.5480
##              Balanced Accuracy : 0.8766
##
##              'Positive' Class : Cercevelik
##

```

Part B: Colours

Colours can be coded via their RGB, three integers between 0 and 255. In this exercise, RGB values were mapped to colour names, but only considering the colour space where $G = 0$.

1)

The `colors_train.csv` dataset was loaded and inspected, it was found that there are five classes in the dataset - red, pink, purple, brown, and blue.

```
# Load the dataset colors_train
df_train <- read_csv('colors_train.csv')

## Rows: 400 Columns: 3
## -- Column specification -----
## Delimiter: ","
## chr (1): color
## dbl (2): r, b
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
df_test <- read_csv('colors_test.csv')

## Rows: 100 Columns: 3
## -- Column specification -----
## Delimiter: ","
## chr (1): color
## dbl (2): r, b
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
# Encode Class variable as factor
colors_train <- df_train %>%
  mutate(color = as.factor(color))
colors_test <- df_test %>%
  mutate(color = as.factor(color))
```

```
# Examine structure of dataframe
str(colors_test)
```

```
## tibble [100 x 3] (S3: tbl_df/tbl/data.frame)
## $ r      : num [1:100] 61 191 224 249 183 49 187 195 161 197 ...
## $ b      : num [1:100] 94 229 51 90 56 74 94 86 76 47 ...
## $ color  : Factor w/ 5 levels "blue","brown",...: 4 4 5 3 5 4 3 3 4 5 ...
```

2)

A QDA algorithm was fit to the classification problem and a confusion matrix was generated.

```

# Fit a QDA algorithm
qda.fit <- qda(color ~ r + b, data = colors_train)
pred.qda.fit = predict(qda.fit, newdata = colors_test)$class

# Compute confusion matrix
confusionMatrix(as.factor(pred.qda.fit),as.factor(colors_test$color), positive = "Yes")

## Confusion Matrix and Statistics
##
##           Reference
## Prediction blue brown pink purple red
##   blue      15     0     0     0     0
##   brown      0    10     0     1     0
##   pink       0     0    22     4     0
##   purple     0     0     0    31     0
##   red        0     0     0     0    17
##
## Overall Statistics
##
##           Accuracy : 0.95
##           95% CI : (0.8872, 0.9836)
##   No Information Rate : 0.36
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.935
##
##   McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: blue Class: brown Class: pink Class: purple
## Sensitivity           1.00         1.0000         1.0000         0.8611
## Specificity           1.00         0.9889         0.9487         1.0000
## Pos Pred Value        1.00         0.9091         0.8462         1.0000
## Neg Pred Value        1.00         1.0000         1.0000         0.9275
## Prevalence            0.15         0.1000         0.2200         0.3600
## Detection Rate        0.15         0.1000         0.2200         0.3100
## Detection Prevalence  0.15         0.1100         0.2600         0.3100
## Balanced Accuracy      1.00         0.9944         0.9744         0.9306
##
##           Class: red
## Sensitivity           1.00
## Specificity           1.00
## Pos Pred Value        1.00
## Neg Pred Value        1.00
## Prevalence            0.17
## Detection Rate        0.17
## Detection Prevalence  0.17
## Balanced Accuracy      1.00

```

3)

Figure 1 visualises the decision boundaries generated by the QDA algorithm.

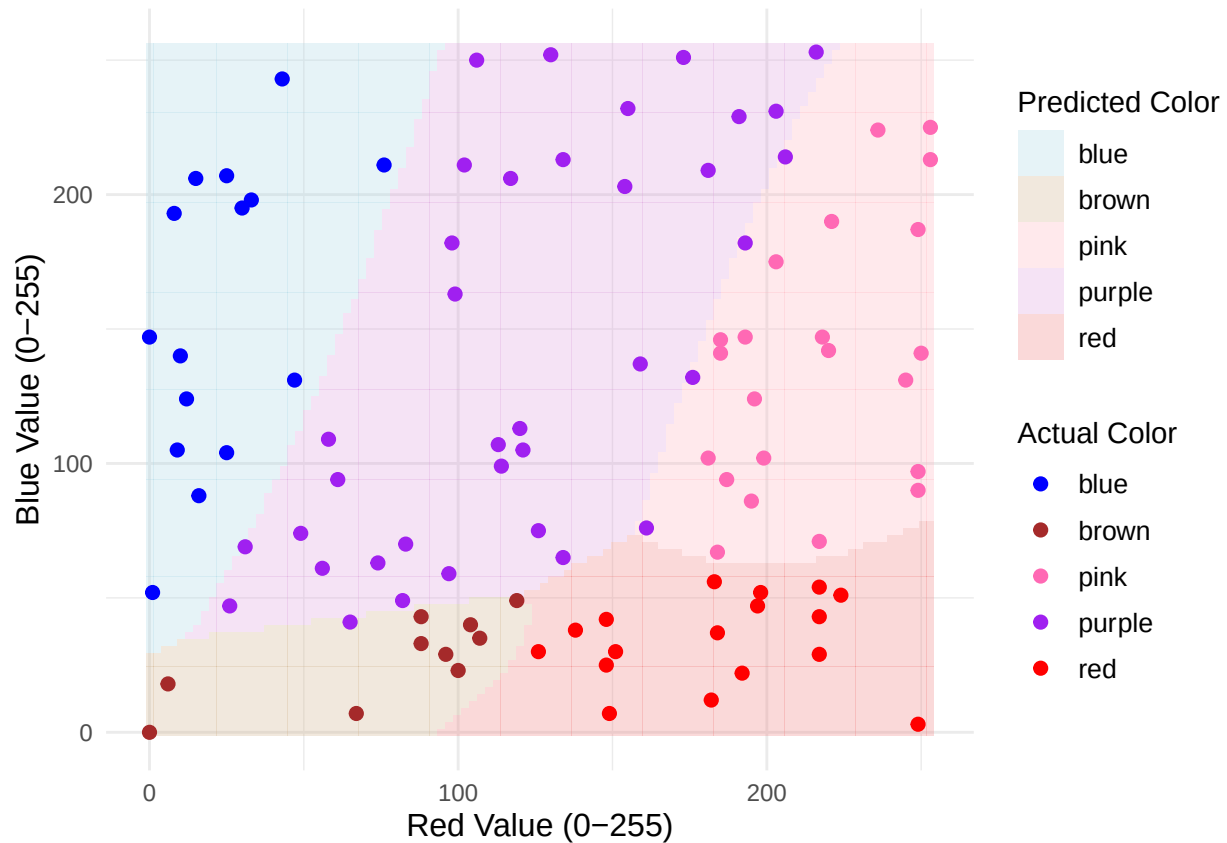


Figure 1: Classification boundaries as defined by the QDA algorithm. The test data and with actual classifications are plotted over the boundaries as points.

4)

The algorithm was tested with the point (200, 0, 200). The algorithm calls this colour 'pink', although the predicted probability of 'purple' is high, suggesting it may lay close to the boundary, which was confirmed by examining Figure 1.

```
# Create a data frame with the test point
test_point <- data.frame(r = 200, b = 200)

# Predict the color class
predicted_color <- predict(qda.fit, newdata = test_point)

# Get the predicted class
predicted_color$class

## [1] pink
## Levels: blue brown pink purple red

# Get the prediction probabilities for all classes
predicted_color$posterior

##          blue          brown         pink         purple          red
## 1 7.14596e-15 2.072491e-35 0.5538038 0.4461962 8.530348e-18
```